

Formal Tool Adoption Audit

Verus, Kani, Aeneas, MPFR through Rug, and Rocq Interval

pid-rs project analysis

24 July 2026

Abstract

This document records an adoption decision for five assurance layers proposed for `pid-rs`. It separates upstream facts from local observations. It records exact release pins, licenses, toolchains, commands, blockers, trust boundaries, negative controls, and permitted claim text. The selected order is: a Rug and MPFR categorical interval oracle, bounded Kani harnesses, a Verus proof pilot for the same pure categorical kernel, an optional Rocq Interval certificate lane, and a deferred Aeneas translation experiment. This document is not evidence that any listed tool verified `pid-rs`. It adds no dependency, CI gate, proof result, certified result, or release claim.

Contents

1	Record status and purpose	2
2	Scope, exclusions, and terms	3
3	Current repository and local state	3
3.1	Repository pins	3
3.2	Local observations	4
4	Decision matrix	4
5	Verus adoption record	5
5.1	Upstream pin and license	5
5.2	Pin and installation rule	5
5.3	First obligations	6
5.4	Trust boundary, blockers, and negative controls	6
5.5	Permitted claim	7
6	Kani adoption record	7
6.1	Upstream pin and license	7
6.2	Pin and installation rule	7
6.3	First obligations	7
6.4	Trust boundary, blockers, and negative controls	7
6.5	Permitted claim	8
7	Aeneas and Charon adoption record	8
7.1	Upstream pin and license	8
7.2	Reproducible experiment rule	9
7.3	Experiment scope and blockers	9
7.4	Permitted claim	9
8	Rug and MPFR adoption record	9
8.1	Upstream pin and license	9
8.2	Future dependency pin	10
8.3	First numerical obligations	10

8.4	Trust boundary, blockers, and negative controls	11
8.5	Permitted claim	11
9	Rocq Interval adoption record	11
9.1	Upstream pin and license	11
9.2	Pin and installation rule	12
9.3	First obligations	12
9.4	Trust boundary, blockers, and negative controls	12
9.5	Permitted claim	13
10	Cross-layer evidence rules	13
10.1	A result stays inside its layer	13
10.2	Required evidence envelope	13
10.3	Fail-closed rules	14
11	Pilot exit criteria	14
12	Conclusion	14
13	Primary source index	14

1 Record status and purpose

Record status: Adoption decision record, dated 24 July 2026.

Evidence status: No tool evaluated in this record has verified `pid-rs`. This record does not report a proof run, model-checking result, certified numerical result, or independent certificate check. It selects bounded pilot work and states the conditions that must apply before a result can become repository evidence.

The audit covers:

1. Verus;
2. Kani;
3. Aeneas with Charon;
4. MPFR through Rug; and
5. Rocq with the Interval package.

The cut-off date is 24 July 2026. Upstream state can change after that date. A pilot must repeat the upstream release check before installation.

The decision is:

1. Pilot a Rug and MPFR categorical reference backend first.
2. Pilot Kani on bounded categorical integer and lattice obligations.
3. Pilot Verus on the same small, pure categorical kernel.
4. Evaluate Rocq Interval after the executable interval format is stable.
5. Defer Aeneas production adoption. Permit one isolated translation experiment.

No one tool replaces Lean. The tools cover different failure classes. Evidence from one layer does not imply evidence from another layer.

2 Scope, exclusions, and terms

This record covers tool fit, versions, licenses, toolchains, installation pins, reproducible commands, trust boundaries, and permitted claim language.

This record does not:

- add a dependency or change CI;
- change a public API, estimator, method catalog, or scientific claim;
- prove Rust-to-specification refinement;
- prove floating-point correctness or statistical calibration;
- prove a theorem about continuous PID;
- approve a license for distribution; or
- claim that the current local installation is a release environment.

License identifiers come from upstream metadata. They are not legal advice. A distribution review is mandatory before code that links to an LGPL or CeCILL-C component is released.

Upstream fact. A fact in an official project repository, release page, manual, or package record.

Local fact. An observation from the workstation on 24 July 2026.

Trust boundary. Code, data, assumptions, or tools that a proof result does not verify. This document also uses the common term trusted computing base, or TCB.

Pilot. A bounded evaluation. It does not mean that a tool is part of the build, CI, or release process.

Certified backend. A new backend that returns an outward interval under a stated rounding contract. It does not mean that an existing binary64 result is certified.

3 Current repository and local state

3.1 Repository pins

Item	Repository pin	Source
Minimum supported Rust version	1.89	Cargo.toml
Lean	4.32.0	audit/formal/lean/lean-toolchain
mathlib	v4.32.0	audit/formal/lean/lakefile.toml
Z3 for the existing algebra checker	4.16.0, 64 bit	scripts/check-z3-pid2-algebra.py

These pins do not apply automatically to a new verifier. Each verifier can have a different compiler and solver pin.

3.2 Local observations

The following commands were run from the repository worktree:

```
rustc --version
cargo --version
(cd audit/formal/lean && lake env lean --version)
z3 --version
pkg-config --modversion mpfr
pkg-config --modversion gmp
command -v <tool>
```

Local item	Observation
rustc	rustc1.96.0(ac68faa202026-05-25)
cargo	cargo1.96.0(30a34c6822026-05-25)
Lean	4.32.0, arm64-apple-darwin24.6.0, commit 8c9756b28d64dab099da31a4c09229a9e6a2ef35
Z3	4.16.0, 64 bit
MPFR found by pkg-config	4.2.2
GMP found by pkg-config	6.3.0
FLINT found by pkg-config	Not found
rug or gmp-mpfr-sys in Cargo.lock	Not present

The commands `verus`, `cargo-verus`, `kani`, `cargo-kani`, `charon`, `aeneas`, `opam`, `rocq`, `rocqchk`, `coqc`, `coqchk`, and `nix` were not found.

This local list is an environment observation only. It is not an installation attestation. Homebrew MPFR availability does not establish a Rust binding, directed-rounding wrapper, or certificate path.

4 Decision matrix

Layer	Decision	First permitted scope	Main reason	Main blocker
Rug with MPFR	Pilot 1	Offline or optional categorical exact-count interval oracle	Direct control of log rounding and atom cancellation	Native C and FFI trust, wrapper review, and LGPL obligations
Kani	Pilot 2	Bounded integer, mask, event, lattice, overflow, and rejection harnesses	Exhaustive bit-precise checks inside declared bounds	Bounds do not generalize; concurrency and transcendental accuracy are out of scope
Verus	Pilot 3	Small pure categorical kernel with explicit contracts	Closest fit for unbounded functional contracts over Rust-like code	Rust 1.96 versus the repository Rust 1.89 minimum; unsupported code and assumptions expand the TCB

Layer	Decision	First permitted scope	Main reason	Main blocker
Rocq Interval	Pilot 4*	Independent checks of fixed emitted interval inequalities	Small kernel checker and analytic interval tactics give a second proof route	Separate ecosystem; the certificate generator and runtime binding remain trusted
Aeneas and Charon	Defer; one experiment	Translation of the same small pure kernel to Lean	Can connect accepted safe Rust to a theorem-prover model	No stable release, Lean 4.31 mismatch, supported-subset limits, and handwritten external models

* Pilot 4 is conditional on a stable, versioned certificate schema.

The order is deliberate. The first pilot gives a numerical oracle that the other pilots can use. Kani and Verus then target the same integer semantics with different proof models. Rocq must not precede a stable certificate schema. Aeneas must not become a release dependency during this evaluation.

5 Verus adoption record

5.1 Upstream pin and license

The selected non-prerelease cut is [release/0.2026.07.18.3a4d30b](#). GitHub records its publication on 20 July 2026.

The release pins:

- Rust 1.96.0;
- `vstd` version 0.0.0-2026-07-12-0122.

The Z3 fetch helper in the tagged source pins Z3 4.12.5.

The license is MIT. At the audit date, the release page had assets for arm64 macOS, x86-64 macOS, x86-64 Linux, and x86-64 Windows. This list is not a general platform support theorem.

Primary pin sources are the [release toolchain](#), [vstd package metadata](#), [Z3 fetch helper](#), and [MIT license](#).

5.2 Pin and installation rule

Do not use a rolling prerelease tag. Do not mix `vstd`, the Verus executable, and solver files from different releases.

For a pilot:

1. Select the platform asset from the immutable release page.
2. Compute and record its SHA-256.
3. Extract it into a version-specific directory.
4. Use `cargo-verus` from that release.
5. Use the solver in the release bundle, or the exact upstream solver pin.
6. Record all tool versions, asset digest, host, and target.

The Cargo metadata and commands are:

```
[package.metadata.verus]
verify = true

cargo verus verify --workspace --locked
cargo verus build --release --locked
```

Verus also supports `--offline`. A release evidence job must use a lockfile and an archived dependency set before it claims offline reproducibility. A focused verification run is a development aid. It is not a substitute for the full verification command.

5.3 First obligations

The pilot can cover:

- multiset preservation by sorting and run-length encoding;
- histogram count totals;
- equality of event counts and event predicates;
- denominator positivity for supported keyed rows;
- canonical source-mask and antichain order;
- Möbius reconstruction; and
- specialized and general categorical-kernel equivalence.

The pilot must not start with logarithms, binary64 error, the complete report API, parallel code, cancellation plumbing, allocators, Python bindings, or continuous estimators.

5.4 Trust boundary, blockers, and negative controls

Verus does not support all Rust code and libraries. The Verus verifier, Rust compiler, LLVM, solver, and platform remain in the trust boundary.

The features `assume`, `external_body`, `external_fn_specification`, and `external` add assumptions or trusted specifications. The pilot must create an assumption inventory. A result with an unreviewed assumption is not accepted.

Verus uses Rust 1.96. The repository supports Rust 1.89. A successful Verus run does not prove that the erased program builds at Rust 1.89. The normal Rust 1.89 gates remain separate.

Required negative controls must:

- mutate an event union to an intersection;
- mutate a source mask;
- remove a Möbius predecessor;
- add an unapproved assumption; and
- run the erased source under Rust 1.89.

The first three mutations must fail a contract. The assumption mutation must fail policy. The MSRV run must detect incompatible erased source.

5.5 Permitted claim

Verus proved the stated contracts for the opted-in functions under the recorded assumptions and pinned Verus, solver, and toolchain. It did not verify rustc or LLVM, unchecked specifications or external bodies, callers outside those contracts, or floating-point transcendental correctness.

Do not state that Verus verified `pid-rs`, the Rust binary, numerical correctness, or statistical correctness.

6 Kani adoption record

6.1 Upstream pin and license

The selected stable release is `kani-0.67.0`, published on 16 January 2026. It pins Rust `nightly-2025-11-21` and CBMC 6.8.0. Kani is dual licensed under MIT or Apache-2.0.

Primary pin sources are:

- [Rust toolchain](#);
- [dependency pins](#);
- [MIT license](#); and
- [Apache-2.0 license](#).

The official easy-install matrix lists x86-64 Linux with GNU libc, x86-64 macOS, and arm64 macOS. The installation guide states Rust 1.58 or newer through `rustup`.

6.2 Pin and installation rule

Use a version-specific `KANI_HOME`:

```
export KANI_HOME=/absolute/tool-cache/kani/0.67.0
cargo install --locked kani-verifier --version 0.67.0
cargo kani setup
cargo kani --version
```

Archive or hash the resulting tool directory. Record the package checksum, all versions, host, target, harness, input bounds, unwind bound, and unwinding assertion result.

```
cargo kani --harness <HARNESS_NAME> --unwind <BOUND>
```

6.3 First obligations

Use Kani for bounded checks of no panic, no out-of-bounds index, no checked integer overflow, resource-estimate arithmetic, mask and lattice indexing, malformed-shape rejection, cancellation paths, specialized and general kernel equivalence, and exact reconstruction identities.

Each harness must declare all shape and count bounds. It must retain an unwinding assertion.

6.4 Trust boundary, blockers, and negative controls

A result for bound N does not imply a result for $N+1$. Kani does not model supported concurrent execution as a concurrent proof. Its floating-point support includes partial or overapproximated intrinsics. It is not a precision proof route for logarithms.

The Kani compiler, CBMC, solver backend, harness assumptions, and environment models remain in the trust boundary.

Required negative controls must:

- reduce an unwind bound and trigger its assertion;
- introduce an index error and obtain a counterexample;
- remove an overflow check and obtain a counterexample;
- change a specialized mask and obtain an equivalence counterexample; and
- label one case outside the proved bound as not covered.

6.5 Permitted claim

Kani exhaustively checked this harness within the declared input and unwind bounds using Kani 0.67.0 and CBMC 6.8.0. It does not establish correctness outside those bounds, concurrent execution semantics, or numerical precision of transcendental functions.

Do not state that Kani proved the algorithm for all inputs, verified parallel execution, or proved the accuracy of `ln` or `digamma`.

7 Aeneas and Charon adoption record

7.1 Upstream pin and license

Aeneas had no stable release at the audit date. Its release page contained nightly prereleases. This record therefore uses:

- Aeneas commit [6e167c9b63a4dafd66d0e0edd9d94669f957ff7b](#);
- Charon commit [527ea8e3b5dcb52edd6aef0f7bc34cc09c11dd59](#);
- Charon Rust nightly [nightly-2026-06-01](#);
- Lean 4.31.0;
- mathlib [v4.31.0](#); and
- OCaml switch example 5.3.0.

Aeneas and Charon are licensed under Apache-2.0.

Primary pin sources are:

- [repository](#);
- [release page](#);
- [Charon pin](#);
- [Charon toolchain](#);
- [Charon Apache-2.0 license](#);
- [Lean toolchain](#);
- [mathlib pin](#); and
- [license](#).

7.2 Reproducible experiment rule

Use a detached full commit. Do not use a moving nightly name as evidence identity.

```
git clone https://github.com/AeneasVerif/aeneas.git
git -C aeneas checkout --detach 6e167c9b63a4dafd66d0e0edd9d94669f957ff7b
cd aeneas
opam switch create 5.3.0
eval "$(opam env)"
make setup-charon
make
```

`makesetup-charon` must resolve the exact Charon commit. Export the complete OPAM switch. Retain `flake.lock` if Nix is used.

```
charon cargo --preset=aeneas
./bin/aeneas -backend lean [OPTIONS] <LLBC_FILE>
```

Retain both commits, Charon nightly, OPAM export, Lean and mathlib pins, translation flags, source digest, LLBC digest, generated Lean digest, and Lean checker output.

7.3 Experiment scope and blockers

Translate only the same small pure kernel selected for Verus. Compare the generated statement with the handwritten Lean specification. The experiment must identify rejected constructs, handwritten external models, theorem correspondence, and toolchain compatibility.

Aeneas supports a subset of safe Rust. Unsafe code and concurrency are out of scope. Some loop control patterns and library operations are unsupported. Handwritten models become trusted inputs.

The backend pins Lean and mathlib 4.31.0. The repository pins 4.32.0. This mismatch blocks direct integration.

Charon translation, Aeneas translation, handwritten models, theorem correspondence, Lean, and the compiler path remain in the trust boundary.

Required negative controls must submit an unsupported construct, mutate an event predicate, remove a handwritten model, alter the Charon checkout, and compare Lean 4.31.0 with 4.32.0. Rejection must be explicit. A compatibility experiment must not be reported as toolchain equivalence.

7.4 Permitted claim

Aeneas translated the accepted safe-Rust subset at the pinned Aeneas and Charon commits, and Lean checked the generated model's stated theorem. This does not establish translation support for all Rust, correctness of handwritten external models, unsafe or concurrent code, or binary equivalence unless those links are separately proved.

Do not state that Aeneas verified the Rust implementation, that generated Lean proves the compiled binary, or that the translation covers unsupported library code.

8 Rug and MPFR adoption record

8.1 Upstream pin and license

The selected pins are:

- Rug 1.30.0, published on 10 July 2026;

- `gmp-mpfr-sys` 1.7.1;
- MPFR 4.2.2;
- GMP 6.3.0; and
- MPC 1.4.1 only if a selected feature requires it.

Rug requires Rust 1.85 or newer. `gmp-mpfr-sys` requires Rust 1.71 or newer. Both fit the repository Rust 1.89 minimum.

Rug, `gmp-mpfr-sys`, and MPFR use LGPL-3.0-or-later licensing in the cited package records. GMP 6.3.0 is available under LGPLv3 or GPLv2. A distribution review is mandatory.

Primary sources are:

- [MPFR current release](#);
- [MPFR manual](#);
- [Rug 1.30.0 metadata](#);
- [Rug rounding modes](#);
- [gmp-mpfr-sys metadata](#); and
- [GMP copying conditions](#).

MPFR states that most functions are correctly rounded as if computed at infinite precision and then rounded in the requested direction. `mpfr_log` computes the natural logarithm. `MPFR_RNDD` rounds toward negative infinity. `MPFR_RNDU` rounds toward positive infinity. Experimental faithful rounding `MPFR_RNDF` must not be used for a certificate.

8.2 Future dependency pin

This record does not add a dependency. A future pilot must use:

```
[dependencies.rug]
version = "=1.30.0"
default-features = false
features = ["integer", "float", "std"]
```

Do not enable experimental `use-system-libs` for reproducible evidence. Use bundled GMP and MPFR sources. Record Cargo checksums and native archive digests. Use an optional offline oracle or a separate small crate. Do not silently replace the stable binary64 API.

8.3 First numerical obligations

For categorical SxPID:

1. Keep histogram and event counts as exact integers.
2. Form exact rational count ratios.
3. Evaluate each logarithm toward both negative and positive infinity.
4. Apply outward rounding to later arithmetic.
5. Apply the concrete Möbius transform to intervals.
6. Increase precision until a declared width or stability rule applies.

7. Return `Unresolved` if an interval contains zero.
8. Return an ordering ambiguity if $I_{\min}$ candidate intervals overlap.
9. Retain exact input, expression-schema, precision-policy, and output digests.

The schema must distinguish certified positive, certified negative, certified exact zero, unresolved sign, unresolved minimizer order, and resource or precision limit. A midpoint is not a certificate. A narrow interval is not proof of statistical validity.

This first backend does not cover continuous KSG. KSG needs separate proofs for neighbor selection, represented distances, strict-radius semantics, estimator assumptions, and statistical behavior.

8.4 Trust boundary, blockers, and negative controls

MPFR, GMP, native build, Rust compiler, C ABI and FFI, `gmp-mpfr-sys`, Rug, wrapper code, expression construction, and data identity remain trusted. The `forbid(unsafe_code)` attribute in `pid-core` does not verify native transitive dependencies.

Required negative controls must reverse one endpoint direction, retain an unresolved near-zero atom, retain an unresolved minimizer order, report a precision limit, bind a count mutation to a new digest, and compare small tables with an independent high-precision implementation.

8.5 Permitted claim

The certified backend returns an outward enclosure of the exact-real expression under MPFR 4.2.2's directed-rounding contract and the audited wrapper. It does not prove the existing binary64 result, the Rust compiler or FFI, or the MPFR implementation itself.

Do not state that MPFR formally verified the result, that the current `f64` output is certified, or that an interval proves estimator calibration.

9 Rocq Interval adoption record

9.1 Upstream pin and license

The selected stable pins are Rocq 9.2.0, released on 27 March 2026, and `coq-interval` 4.11.4, published on 23 February 2026. Rocq 9.3 release-candidate builds are prereleases and are not selected.

Rocq core is LGPL-2.1-only. `coq-interval` is CeCILL-C.

The package archive SHA-512 for `coq-interval` 4.11.4 is:

```
5b2ccff32c3d6caa9002455dc472d6c4c8f70337581d038ca432dd92ed90b262
b203960a2a62896c12992be2ff2436bf7e6a8ffd3aec625859f47088d262ef00
```

The two lines are one continuous digest.

Primary sources are:

- [Rocq site](#);
- [Rocq 9.2.0 release](#);
- [core package metadata](#);
- [Interval package record](#);

- [Interval versions](#);
- [Interval documentation](#);
- [exact OPAM metadata](#);
- [Rocq OPAM instructions](#); and
- [compile and check commands](#).

The exact OPAM metadata permits the modern split `coq-core` and `coq-stdlib` route. It also requires Flocq, MathComp, Coquelicot, bignums, OCaml, and a compiler.

9.2 Pin and installation rule

```
opam switch create 4.14.2
eval "$(opam env)"
opam repo add rocq-released https://rocq-prover.org/opam/released
opam install rocq-prover=9.2.0 rocq-core=9.2.0 coq-interval=4.11.4
opam pin add rocq-core 9.2.0
opam pin add coq-interval 4.11.4
opam switch export --full rocq-9.2.0-interval-4.11.4.export
```

Retain exact OPAM version, repository commit, full switch export, package hashes, certificate source digest, compiled object digest, and checker output with assumptions.

```
rocq compile -q Certificate.v
rocq check -o Certificate.vo
```

The `-o` output exposes the logical context and assumptions. The evidence gate must reject `Admitted`, `admit`, unexpected axioms, and an incomplete check.

9.3 First obligations

Use Rocq Interval only after the interval expression has a versioned schema. Candidate theorems can state:

- a rational logarithmic expression lies inside emitted endpoints;
- an atom interval is strictly positive or negative;
- two $I_{\min}$ intervals are strictly ordered;
- a Möbius reconstruction encloses each cumulative coordinate; and
- a certificate binds to exact count-table and expression-schema digests.

The theorem statement must bind to the exact input identity. A theorem about the wrong count table can still be a valid theorem.

9.4 Trust boundary, blockers, and negative controls

Rocq's kernel checks the proof object. It does not prove that the generator encoded the intended formula or data, that a file digest was bound to a runtime operation, that `pid-rs` used the result, or that the statement has statistical meaning.

The generator, statement schema, input binding, serialization code, OCaml runtime, Rocq kernel, and platform remain in the trust boundary.

Required negative controls must mutate an endpoint, mutate the bound input digest, insert an admission, add an unexpected axiom, and present a valid proof for a different count table. Each control must fail the applicable proof or identity gate.

9.5 Permitted claim

Rocq’s kernel independently checked the emitted inequality theorem and exposed its remaining assumptions. It does not prove that the generator encoded the intended data and expression or that the `pid-rs` runtime used that certificate.

Do not state that Rocq verified the Rust implementation, proved input provenance, or proved application validity.

10 Cross-layer evidence rules

10.1 A result stays inside its layer

Evidence	What it can support	What it cannot support by itself
Existing Lean theorem	Exact theorem in the declared Lean model	Rust implementation, binary64 output, or data assumptions
Verus proof	Contracts for opted-in supported functions	Compiler, external assumptions, or transcendental accuracy
Kani harness	Exhaustive bounded property	Unbounded inputs, concurrent behavior, or numerical accuracy
MPFR interval	Outward enclosure for one encoded expression	Existing <code>f64</code> , statistical calibration, or an FFI proof
Rocq certificate	Kernel-checked encoded theorem	Correct generator, runtime use, or intended data identity
Hidden benchmark	Observed behavior on a frozen corpus	Universal correctness or a mathematical theorem

Evidence can compose only when a checked bridge binds source revision, tool revision, specification revision, input identity, generated-artifact identity, result identity, assumptions, and claim scope.

10.2 Required evidence envelope

Every accepted pilot result must record:

1. exact tool version or commit;
2. compiler, solver, and backend pins;
3. host, target, and installation asset digest;
4. source commit and dirty-state policy;
5. command line;
6. input bounds or theorem statement;
7. assumption inventory;
8. generated artifact hashes;
9. checker result;
10. negative-control result;
11. unsupported cases; and
12. exact permitted claim text.

A green exit code without this envelope is a development result. It is not release evidence.

10.3 Fail-closed rules

The process must return a typed failure or unresolved result if a pin differs, a digest is absent, a solver or compiler version is unknown, an unwinding assertion fails, an assumption is not approved, an artifact digest differs, an interval does not determine the requested result, a certificate has an admission or unexpected axiom, or a supported-subset translator rejects the source.

The process must not change a failure into an approximate success.

11 Pilot exit criteria

Rug and MPFR. All endpoints must be outward rounded. Precision escalation must have a typed result. Ambiguous signs and ties must stay unresolved. Exact identities and independent small-table checks must be retained. A license review must define the distribution route.

Kani. Every harness must declare bounds. Every loop must have a passing unwinding assertion. Meaningful mutations must produce counterexamples. No harness can model transcendental accuracy. The report must state the bounded domain.

Verus. The complete selected kernel must verify. The assumption inventory must be empty or approved. Semantic mutations must fail. The erased program must pass Rust 1.89 gates. Ordinary tests must agree with specification fixtures.

Rocq Interval. The schema must be versioned. `rocqcheck-o` must report no unexpected assumption. The gate must reject admissions. Identity mutations must fail. Archived exact inputs must regenerate the certificate.

Aeneas. The selected subset must translate without an unsound workaround. External models must be minimal and reviewed. The theorem correspondence must be documented. Lean compatibility must be resolved. Source mutations must affect the model and proof.

12 Conclusion

The most useful immediate addition is an executable categorical interval oracle with explicit unresolved states. Kani can then attack bounded compiled behavior. Verus can attack unbounded functional contracts for the same small kernel.

Rocq Interval is useful only when an independent certificate has operational value and the generator-to-statement bridge is controlled. Aeneas is a valuable research route, but its current release and toolchain state does not justify production adoption.

`pid-rs` is not end-to-end formally verified. Each future result must state the exact layer, theorem or bound, assumptions, tool pins, trust boundary, and unsupported claims.

13 Primary source index

Verus

- [Release 0.2026.07.18.3a4d30b](#)
- [Verus guide](#)
- [Cargo integration](#)

- [Trusted computing base](#)
- [Assumption specifications](#)

Kani

- [Release 0.67.0](#)
- [Installation guide](#)
- [Kani manual](#)
- [Loop unwinding](#)
- [Rust feature support](#)
- [Intrinsic support](#)

Aeneas and Charon

- [Aeneas repository](#)
- [Aeneas releases](#)
- [Pinned Aeneas commit](#)

MPFR and Rug

- [MPFR project](#)
- [MPFR 4.2.2](#)
- [MPFR manual](#)
- [Rug 1.30.0](#)
- [gmp-mpfr-sys 1.7.1](#)
- [GMP copying conditions](#)

Rocq and Interval

- [Rocq project](#)
- [Rocq 9.2.0](#)
- [coq-interval 4.11.4](#)
- [Interval documentation](#)
- [Rocq OPAM instructions](#)
- [Rocq compile and check tools](#)